

A Hitchhiker's Guide to Apache Kafka Data Migrations

Current London 2026

Michael Muehlbeyer

whoami

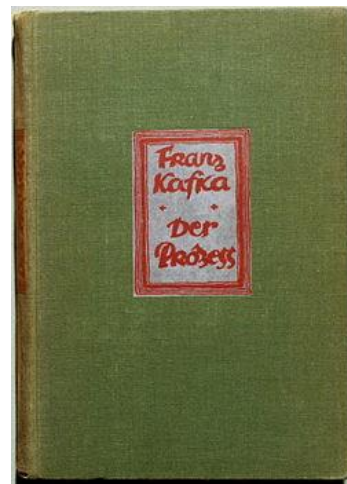


- Senior Principal Consultant @Apps Associates
- Focssing on data platforms
 - PostgreSQL, Oracle DB and other RDBMS
 - Kafka & Streaming
- At home in Heilbronn region, BW, Germany
- 🖥️, 📚, 📡 keeping, 🎧, 🎧



Agenda

- Why migrate
- How to Migrate
- Pitfalls
- Best practices

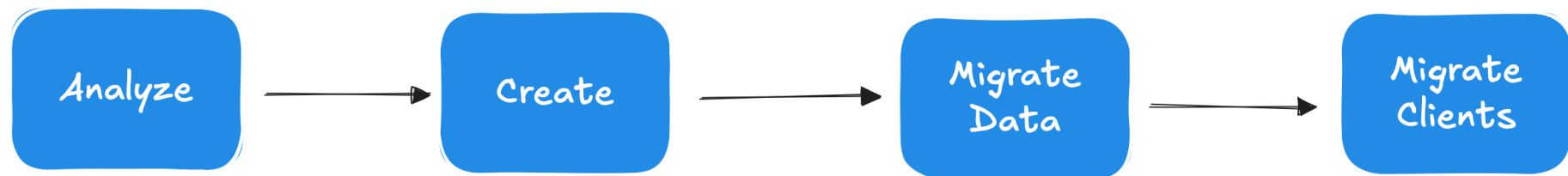


Why migrate?

- Hardware
 - Performance
 - Outdated Hardware
 - Storage limits
- Software
 - OS EOL
 - Kafa EOL
- Cloud Shift
 - Save operational cost
- **Zookeeper EOL**
- ...

How to migrate?

How to migrate



How to migrate – What?

- Topics
 - Each and every topic needed?
 - Get rid of no longer needed data
- Schemas
- ACLs
- User
- Roles
- Certificates

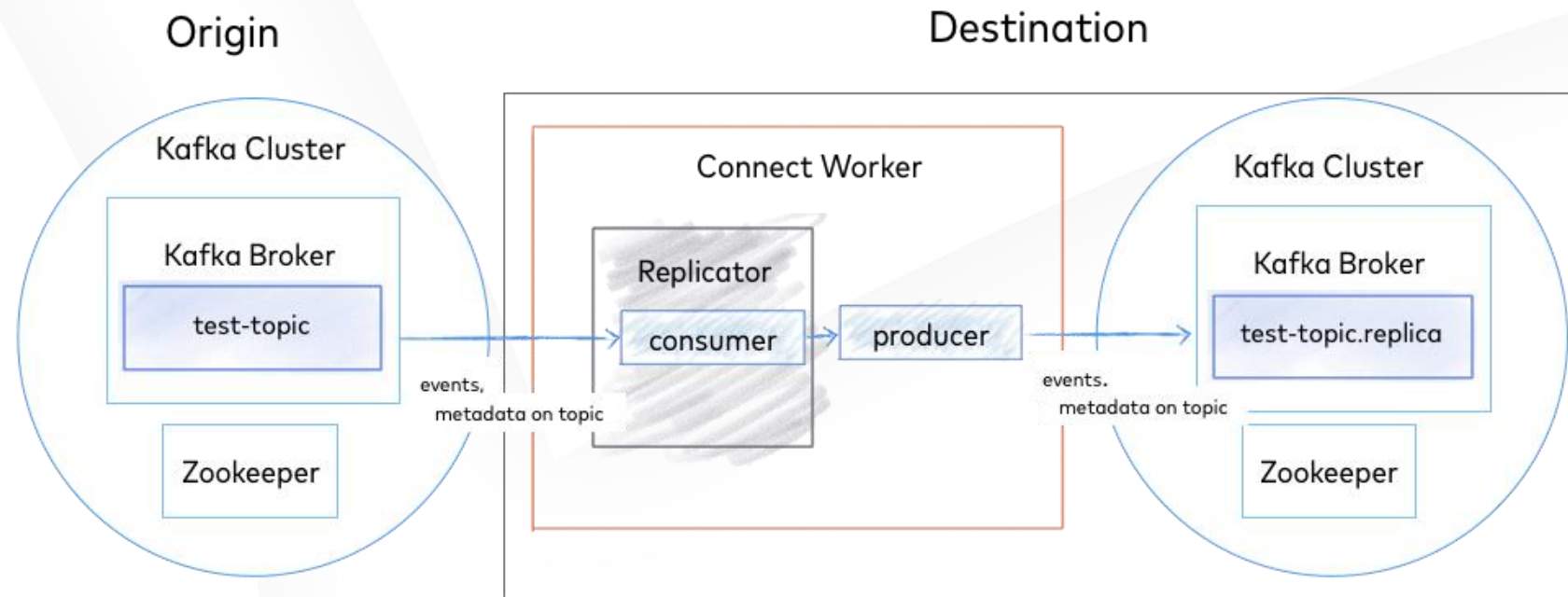
How to migrate? - Options

- Confluent Replicator/Cluster Link
- Flink
- Self-coded Apps
- Kafka Copy Paste (KCP)
- MirrorMaker2/KafkaConnect

Confluent Replicator

- Reads the topics of the source cluster
- Sends them to a new cluster
- License needed
 - Support available 😊
- Confluent platform needed to make use of offset translation feature

Confluent Replicator



Cluster Linking

- Topic 1:1 migration
- Migration 1 by 1 or in a batch possible
- Enterprise license needed
 - Support available
- Only Confluent platform/Cloud as target support
- Battle proof

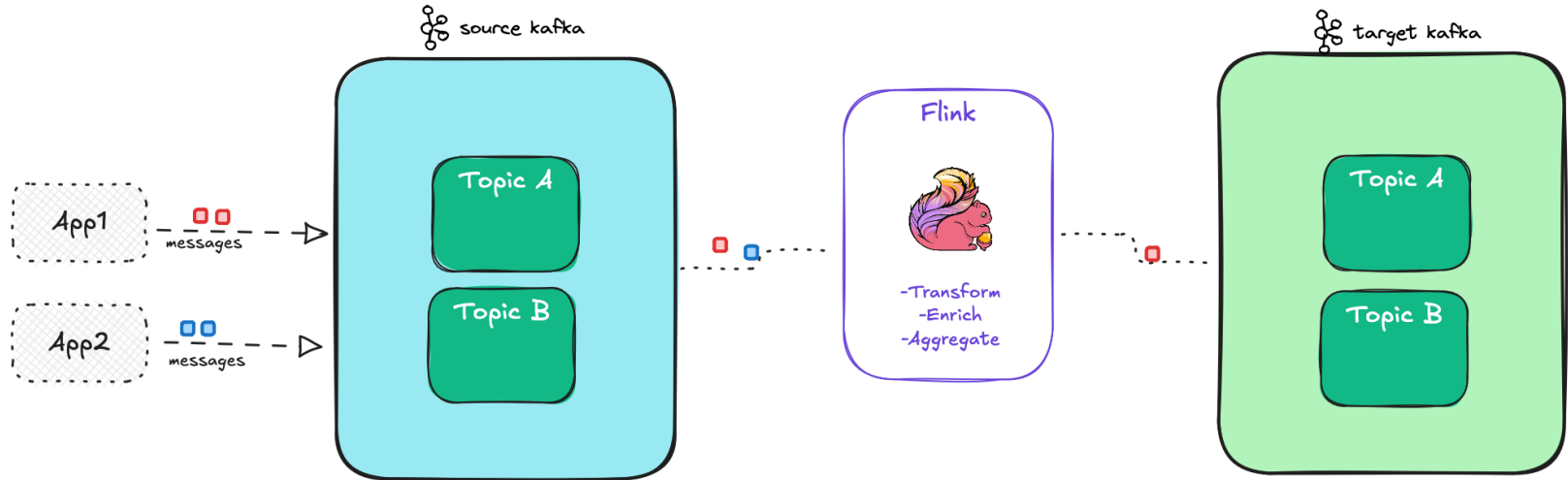
Cluster Linking



Apache Flink

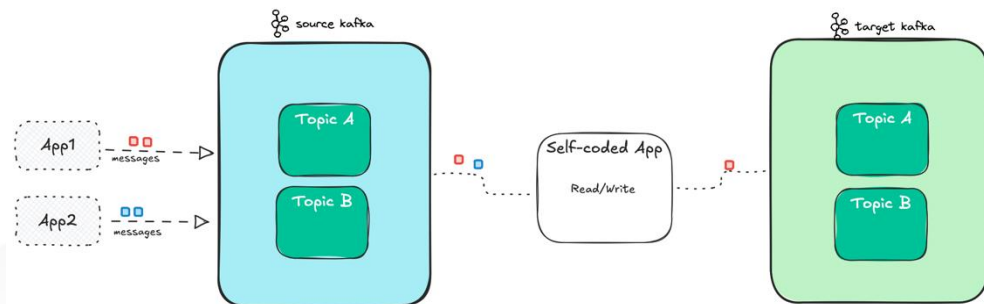
- Read the topics of the source cluster
- Send them to a new cluster
- Use the Flink Kafka connector
- Do some transformations if needed
 - Heads up!
- Create the topic in the new cluster upfront
 - Take care of configs, ACLs,...

Apache Flink

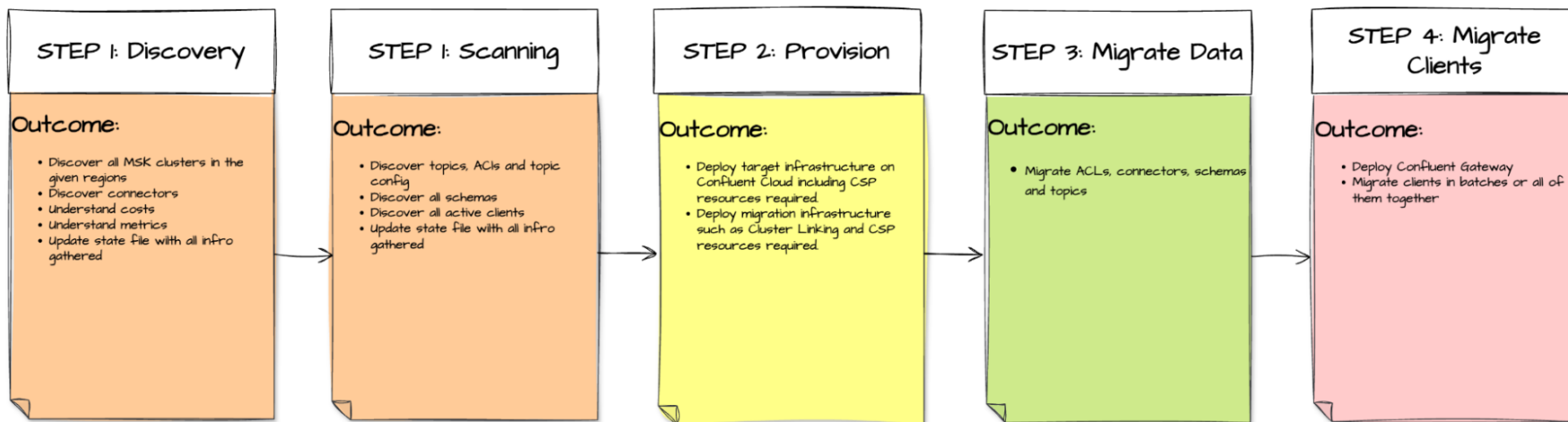


Self-coded Apps

- Create a self-coded app
- Migrate data via classical producer/consumer, Kafka streams app, ...
 - Choose the stack you're familiar with
- Take care of topic creation, ACLs, Schema and so on



Kafka Copy Paste (KCP)



Source <https://www.confluent.io/blog/automate-kafka-migration-with-kcp/>

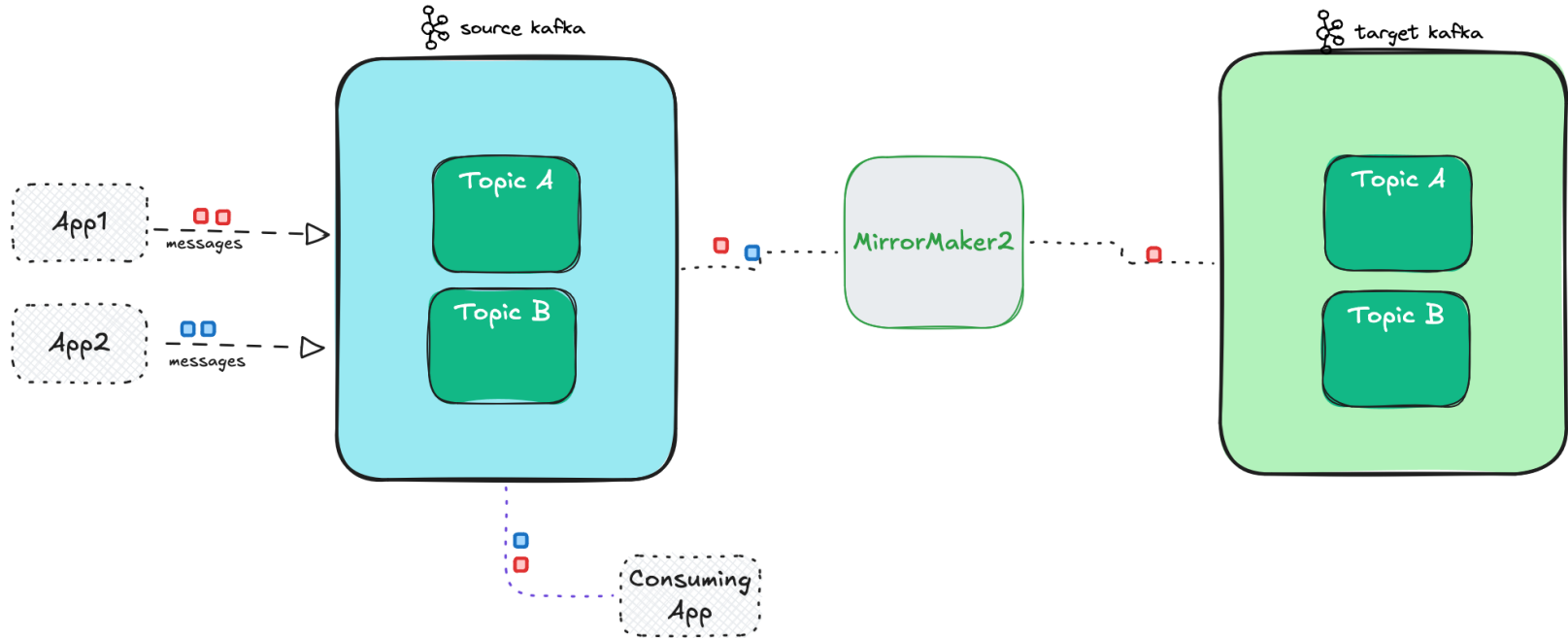
Kafka Copy Paste (KCP)

- Easy scanning of the existing env
 - Clusters (Topics,
 - Schemas
 - Connectors
- Automatically deploys migration infrastructure
- Cluster links are created automatically between existing and new Confluent Cloud cluster
- Confluent Cloud as target
- AWS source only as of now

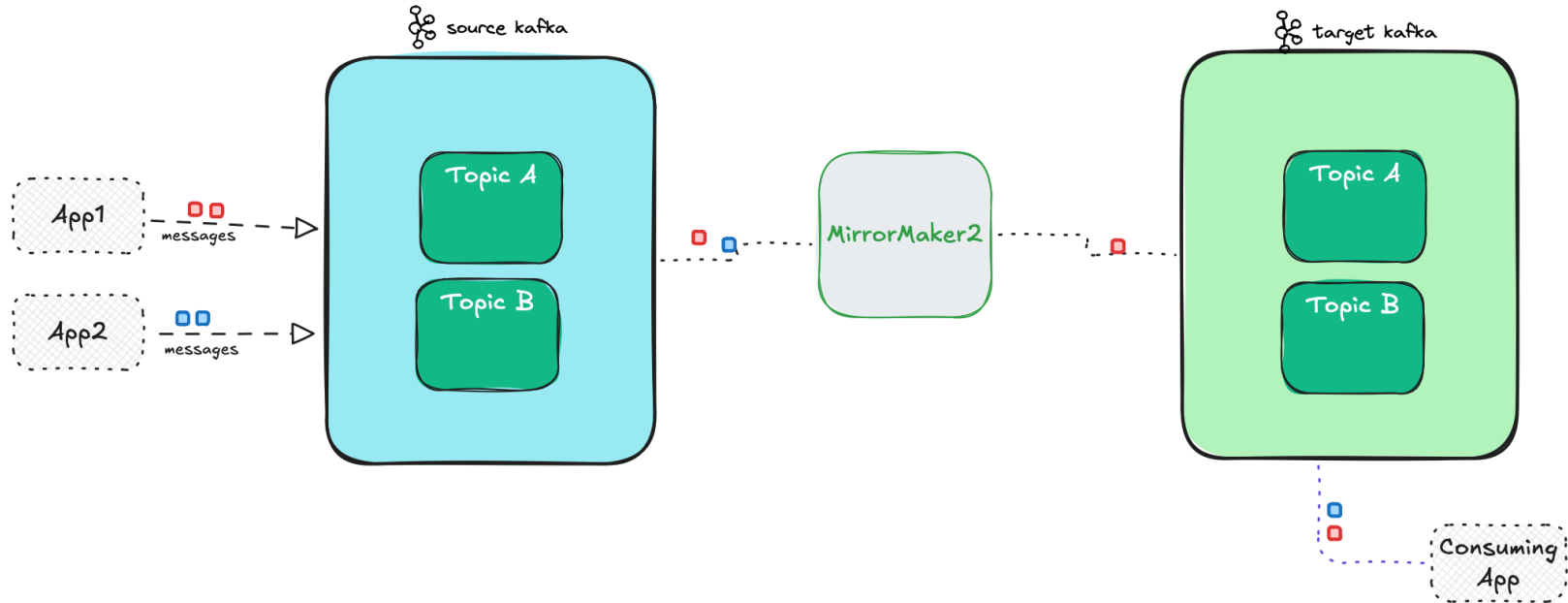
MirroMaker2

- MirrorMaker2(MM2) creates a 1:1 topic copy from source to target cluster
- MM2 takes care of copying
 - ACLS (`sync.topic.acls.enabled`)
 - Configuration
 - Emits offset-sync to internal topics
- Built on top of Kafka Connect
 - Basically a connector

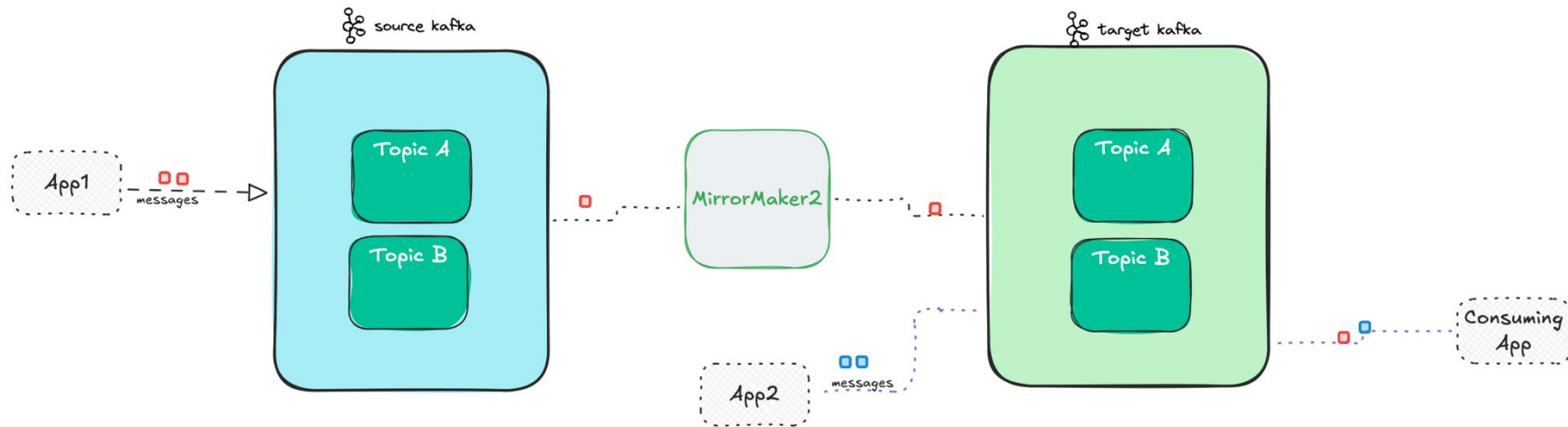
MirrorMaker2



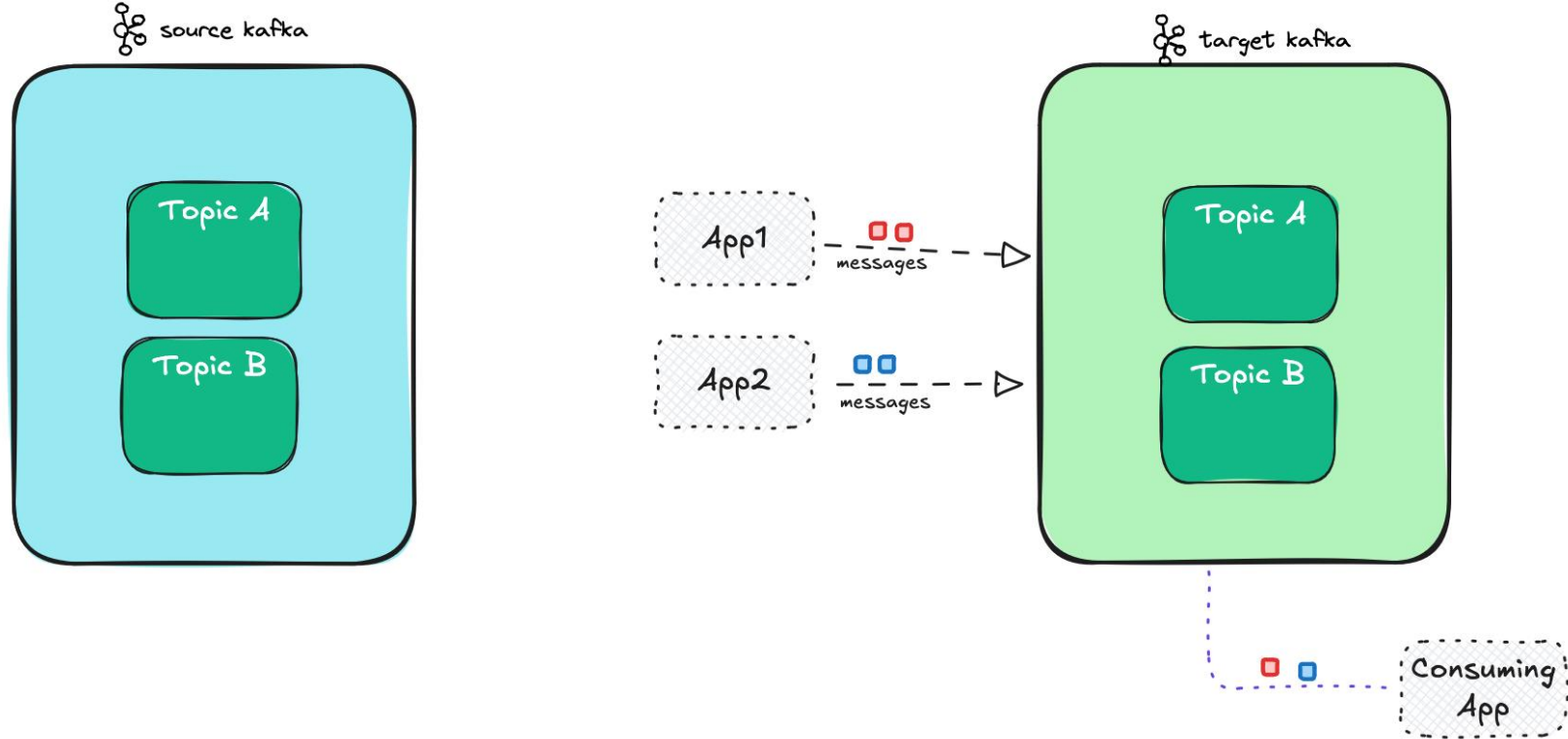
MirrorMaker2



MirrorMaker2



MirrorMaker2



What else?

What else?

- Schemas
- Certificates
- Deployment pipelines
- Monitoring
- Backup
- ...

Pitfalls

Pitfalls

- Clients
 - Client compatibility
 - Client connectivity
 - Are you aware of each client? ;-)
- Network connectivity
 - Especially when moving to Cloud
- Security
 - Certificates
 - ACLs
 - (LDAP connectivity)

Pitfalls

- Performance
 - Cloud migration network connectivity
- Offset translation
 - Non Java clients
- Zookeeper vs. Kraft
 - Check the connection strings of clients
- SchemaRegistry
 - Confluent SR, Google SR, self-hosted
 - Authentication?

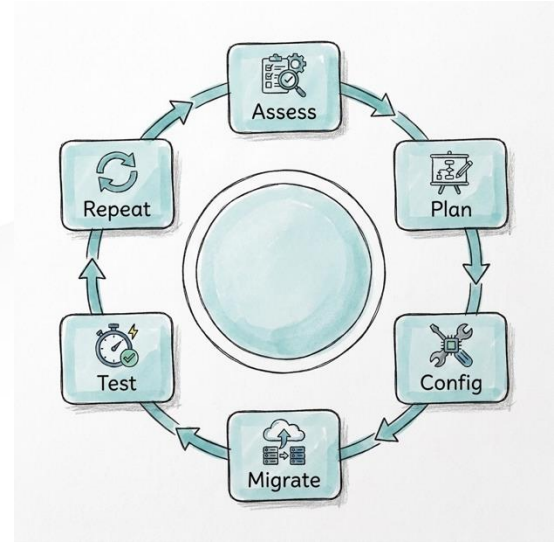
Best Practices

Best practices

- Plan carefully
- Take your time
- Fail fast !
 - Evaluate multiple solutions and dismiss quick if its not working
 - Be brave 😊
- Test and UAT environments needed
 - One env for you
 - One env for the app testers etc.
 - Near prod sizing
- Know the limits of your target env
 - Cloud, OnPrem, ...

assess, plan, config, migrate, test, repeat!

- Take time to assess your current landscape
 - Connected apps
 - Client version, topics, dependencies
 - Get all the necessary people involved
 - Dependencies
 - Other systems or apps
- Plan carefully
 - Create a migration plan
 - Resolve dependencies
 - Check for apps/topic without dependencies
 - Check for low hanging fruits
 - Hot and cold data?
 - Choose the migration tool which fullfills your needs best
 - Probably not a 100% fit



assess, plan, config, migrate, test, repeat!

- Config
 - Setup your test and UAT environment accordingly
 - Script, terraform, ansible → automate the setup
- Migrate
 - Migrate the topic data, schema **and** configs
- Test
 - Test if everything is working fine
 - Test also the dependent systems
- Repeat
 - Repeat the migration until everything works fine
 - Switch your migration tool if needed
 - Documentation
 - create a step by step runbook

42

42

- all the above are general recommendations from the field
 - based on experiences made through the last years
- if you ask me how to migrate "your specific env" the answer will be probably "42" or "it depends"
- Even with a proper strategy according to **assess, plan, config, migrate, test, repeat** you will (probably) face some unexpected issues during the prod migration
 - Try to keep the possibility small with a proper plan



Links

- <https://developer.confluent.io/community/>
- <https://github.com/mmuehlbeyer/kafka-migrations>
- <https://github.com/mmuehlbeyer/kafka-zk-to-kraft>
- <https://github.com/mmuehlbeyer/talks>

- <https://kafka.apache.org/>
- <https://www.confluent.io/blog/automate-kafka-migration-with-kcp/>

A long-exposure photograph of a city at night. In the background, several tall skyscrapers are illuminated with lights. In the foreground, a multi-lane highway is visible, with light trails from cars creating streaks of white and blue. The sky is dark, and the overall scene is lit with a cool blue tone.

Thank you